Ubuntu Terminal Basics

A practical first tour of the Linux terminal

Ubuntu command line essentials

Dr. Kaveh Hooshmandi

Primary academic email: k.hooshmandi@arakut.ac.ir
Secondary email: Hooshmandi.kaveh@gmail.com

What you will learn

- Understand the terminal, Shell, Bash, and user identity
- Read paths; use `pwd`, `cd /`, and `cd ~`
- Create and read files, redirect output, and run programs
- Use completion, permissions, and administrator access carefully
- Build a practical workflow with packages, processes, and tools

One simple idea

The terminal is a direct way to ask Ubuntu to do a task. Learn a few small commands, then reuse them throughout the course.

Who cares about the terminal?

- It gives a consistent way to describe steps in a tutorial.
- Commands are easy to repeat, share, and adapt.
- Development tools are often launched and inspected from a shell.

You do not need to become a terminal expert today.

Think of it as a tool

A graphical interface is great for many tasks. The terminal complements it when exact, repeatable commands are useful.

The terminal: a command workspace

Hint

Open a terminal with `Ctrl+Alt+T`.

The shell reads a command, runs it, then shows the result and a new prompt.

Command shape

`ls -l ~/Documents`

`ls` (command: list contents)

`-l` (option: show details)

`~/Documents` (target folder)

Each command and its meaning

Command	Explanation
`pwd`	Print the current directory's path.
`ls`	List files and folders.
`cd`	Change the current directory.
`mkdir`	Create a directory.
`touch`	Create a file or update its timestamps.
`cat`	Print a file's contents.
`rm`	Delete files (`-r` for directories).
`grep`	Select lines matching a pattern.
`nano`	Edit a text file in the terminal.
`apt`	Manage Ubuntu software packages.
`sudo`	Run a command as root by default.
`alias`	Define a shortcut name for a command.

Shell, Bash, and the root user

Shell: the general idea

A shell reads text commands, interprets them, and runs them. For example, `ls` (lists the current folder's contents).

Bash: one kind of shell

`bash` is a specific shell. Other shells include `sh`, `zsh`, and `fish`.

root: the administrator account

The `root` user has extensive system access. A *root shell* is a shell running as this user.

Warning: Return to your regular user

`sudo -i` (starts a root shell); `exit` (leaves it). Check each command carefully while in a root shell.

A typical terminal prompt

`kaveh@ubuntu:~$` (regular user) `root@ubuntu:~#` (root user).
The prompt can be customized; `whoami` (prints the current user) is the reliable check.

Read the prompt: the shell is ready for your command

A prompt is the text the shell displays when it is ready for your next command:

```
kaveh@ubuntu:~$
```

Each part and its meaning

`kaveh`	(the current username).
`ubuntu`	(the computer's name, or hostname).
`~`	(the current folder is this user's Home).
`$`	(the usual prompt symbol for a regular user).

Now type a command

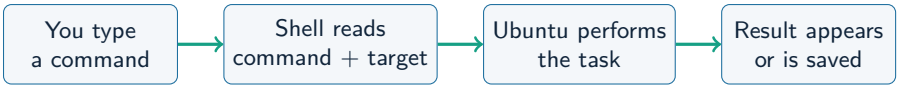
```
kaveh@ubuntu:~$ ls
```

Type only `ls` (list files), then press `Enter`.

Hint

A root shell commonly ends its prompt with `#`. Prompts can be customized; `whoami` (prints the current username) lets you check your identity.

How a terminal command gets work done



Example: ask for your location

Type `pwd`. Ubuntu prints the full path of the current folder.

Hint

Most commands finish and return a prompt. A running program can be stopped with `Ctrl+C`.

Useful command line programs (1/2)

Commands and practical examples

Command	Example	What to expect
`pwd`	`pwd`	Prints the full path of your current folder.
`mkdir`	`mkdir a_folder`	Creates a new folder here.
`cd`	`cd a_folder`	Moves the shell into that folder.
`touch`	`touch test.txt`	Creates an empty file, or updates its timestamp.
`cat`	`cat test.txt`	Prints the contents of a text file.
`rm`	`rm test.txt`	Deletes a file; `-r` also removes a directory tree.

Warning: Know your location before `rm`

Run `pwd` and `ls` first. A deleted file normally does not go to the Trash.

Useful command line programs (2/2)

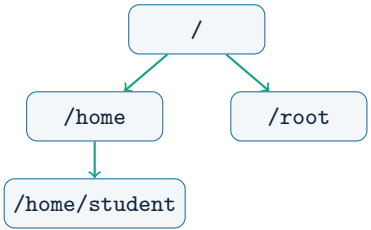
Commands and practical examples (continued)

Command	Example	What to expect
`ls`	`ls`	Lists files and folders.
`grep`	`cat test.txt | grep robot`	Keeps lines that contain `robot`.
`nano`	`nano test.txt`	Opens a simple terminal text editor.
`sudo`	`sudo apt update`	Runs a command with administrator rights.
`apt`	`sudo apt install git`	Installs an Ubuntu package (after repository setup).
`alias`	`alias hi="echo hello"`	Creates a short name for a command in this shell.

Paths first: filesystem root, Home, and here

Read these path symbols

- `/` (the top of the filesystem tree).
- `~` or `~/` (the current user's Home folder).
- `.` (the current working folder).
- `..` (the parent folder).



Hint

`/` names the filesystem root; `root` names the administrator account; `/root` is its usual Home folder. For user `student`, Home is usually `/home/student`.

Find your location, go to /, then return Home

```
pwd
cd /
pwd
cd ~
pwd
```

Run one command per line

1. `pwd` (prints your current working directory).
2. `cd /` (moves to the filesystem root).
3. `pwd` (now prints `/`).
4. `cd ~` (returns to your Home folder).
5. `pwd` (for user `student`, typically prints `/home/student`).

Hint

`cd` means change directory; `pwd` means print working directory. Both `cd` with no argument and `cd ~/` also take you Home. Changing folders keeps the same user identity.

Make a practice folder

```
# Move to your home folder, then create a workspace for this exercise
cd ~
mkdir terminal_practice
cd terminal_practice
touch test.txt
pwd
ls
```

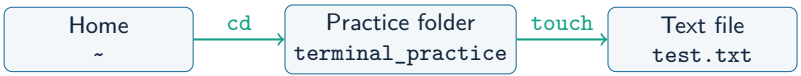
Hint

`~` means your home folder. `.` means the folder you are in now.

What should you see?

`pwd` ends in `/terminal_practice`; `ls` lists `test.txt`. The prompt returns after each command.

Folders are locations: read the path



Absolute, relative, and Home-relative paths

`/home/student/terminal_practice` is an absolute path (it starts at `/`). From Home, `cd terminal_practice` uses a relative path (it starts here); `cd ~/terminal_practice` starts from Home even if you are elsewhere.

Check before changing files

`cd ..` moves to the parent folder; `cd .` stays in the current folder. Run `pwd` to confirm your location and `ls` to inspect its contents.

First file: echo, cat, and output redirection

```
echo "first line" > test.txt
echo "second line" >> test.txt
cat test.txt
ls > test.txt
```

In your practice folder: read the commands in order

1. `echo "first line" > test.txt` (creates `test.txt`, or replaces its old contents).
2. `echo "second line" >> test.txt` (adds a second line without removing the first).
3. `cat test.txt` (prints `first line` followed by `second line`).
4. `ls > test.txt` (replaces the two lines with the current folder's listing).

Warning: The two redirection signs do different things

> (overwrite) and >> (append). Check the target filename before pressing Enter.

Create and run a Python file

Try these commands

```
echo 'print("Hello")' > test.py
python3 test.py
```

`echo` (prints the code); `>` (saves it to `test.py`); `python3 test.py` (runs the file and prints `Hello`).

Hint

Continue in `~/terminal_practice`. Run `pwd` (check your location) and `ls` (check the files) before running the script. Python 3 must be installed.

Create, edit, and read a text file

```
nano test.txt
```

Type a short line, then save and exit Nano:

1. `Ctrl+X`
2. `Y` to save
3. `Enter` to confirm the filename

```
cat test.txt
ls
```

What to expect

`cat test.txt` prints the line you saved. `ls` lists `test.txt` in the current folder.

Terminal keyboard shortcuts

Copy and paste in a terminal

Copy selected text: `Ctrl+Shift+C`

Paste text: `Ctrl+Shift+V`

Hint

Nano displays shortcuts along the bottom. Its symbol `^X` means `Ctrl+X`.

Stop a running command

```
sleep 30
```

Press `Ctrl+C` to stop it and return to the prompt.

Clean up the practice folder carefully

Warning: Pause before using `rm`

Files removed with `rm` do not go to the Trash. Check the path and target first.

```
# Go back home, then remove only the practice folder from this example
cd ~
  rm -r terminal_practice
```

- Use `ls` and `pwd` to verify your location.
- Never paste an unfamiliar destructive command without understanding its target.

Search text and reuse a command

Find matching names with grep

```
ls | grep test
```

`ls` lists filenames; the pipe passes them to `grep`, which keeps names containing `test`.

Create a short alias

```
alias hi='echo hello'
hi
```

`alias` gives a command a short name in the current shell session.

Hint

A pipe | connects one command's output to the next command's input.

Tab completion: type less, check more

Try it

1. Type `cd ~` and press Enter.
2. Type `ls` to see the available names.
3. Start a command, such as `cat result_`.
4. Press `Tab` to complete a unique match.

If more than one name matches

Press `Tab` twice to display possible matches. Add another letter or two, then press Tab again.

Hint

Tab completion works for commands, folders, and files. Use it often to avoid long-path typos.

Create, copy, move, and remove files

```
cd ~/linux_lab
mkdir -p backup archive
touch test.txt
cp test.txt backup/test.txt          # copy to backup
mv backup/test.txt archive/test.txt # move the copy
ls archive
rm archive/test.txt                  # remove the copy
```

Predict the listing

`ls archive` shows the copied `test.txt`. After `rm`, only the original in `linux_lab` remains.

Warning: Check the exact target

`rm` acts immediately. Use `pwd` and `ls` before removal. A directory needs `rm -r`; never use that option casually.

Find files and use patterns carefully

```
cd ~/linux_lab
find . -name '*.txt'        # search this tree
ls *.txt                    # list matching names here
```

What the star means

In `ls *.txt`, Bash expands `*.txt` to matching filenames before `ls` runs. In `find`, the quoted pattern is passed to the program.

Warning: Preview before deleting

If `ls *.txt` prints `test.txt`, then `rm *.txt` deletes it. If several text files match, it deletes all of them. Use `rm test.txt` only after confirming that exact target.

Install Ubuntu packages with apt

```
sudo apt update      # refresh package lists
apt search git       # search available packages
apt show git         # inspect version and description
sudo apt install git # install the selected package
```

Understand the sequence

`update` refreshes the package index; `install` changes the system. `sudo` is needed for system package changes, so read the package name first.

Hint

`apt list --installed git` checks whether `git` is installed. Package availability depends on configured repositories.

Environment variables: settings in a shell

```
echo "$HOME"                          # home directory
echo "$PATH"                          # program search path
mkdir -p "$HOME/My Project"
export PROJECT_DIR="$HOME/My Project"
cd "$PROJECT_DIR"
pwd
```

Why are there quotation marks?

Quotes keep the space in `My Project` inside one path. `cd "$PROJECT_DIR"` passes that full path as one argument to `cd`; `pwd` displays the location, for example `/home/kaveh/My Project`. `export` makes `PROJECT_DIR` available to programs started by this shell; a separate terminal does not inherit it.

Shell scripts: .sh files, bash, and source

A `.sh` file is a text file containing Shell commands that run in order.

Example file: `demo_env.sh`

```
export DEMO_MODE=ready
echo "Hello"
```

Run it in a separate Bash process

```
bash demo_env.sh
```

Prints `Hello`. Its exported variable does not remain in the current shell.

Run it in the current shell

```
source ./demo_env.sh
echo "$DEMO_MODE"
```

Prints `Hello`, then `ready`. The variable remains available in this shell.

Hint

Bash reads `~/.bashrc` when an interactive shell opens. Add only startup commands you intend to run each time.

Permissions: read, write, and execute

```
cd ~/linux_lab
printf '#!/usr/bin/env bash\necho ready\n' > check.sh
ls -l check.sh
chmod u+x check.sh
./check.sh              # expected: ready
```

Read `chmod u+x`

`u` means the file's owner; `+x` adds execute permission for that owner. `./check.sh` runs the script in the current folder.

Shebang: which program runs a script?

Inside `check.sh`

```
#!/usr/bin/env bash
echo ready
```

The first line is a *shebang*. Linux starts `env`, which finds Bash in `PATH`. Bash reads `echo ready` and prints `ready`.

Run the file directly

```
./check.sh
```

This uses the shebang. First make the file executable with `chmod u+x check.sh`.

Name Bash yourself

```
bash check.sh
```

Bash reads the file directly. The shebang is a comment; the file needs read permission, but not execute permission.

Hint

For Python code, use `#!/usr/bin/env python3` and write `print("ready")`. Bash scripts can use `echo`, `export`, `if`, and `for`.

Processes: run and stop a node

```
ros2 run turtlesim turtlesim_node   # foreground: press Ctrl+C
ros2 run turtlesim turtlesim_node & # background: prompt returns
jobs                                # jobs in this terminal
kill %1                             # stop job 1 (check jobs first)
```

Foreground and background

The first command holds this terminal; `Ctrl+C` is the usual way to stop it. With `&`, the prompt returns. `jobs` lists jobs of this shell; use the number it shows, such as `%1`.

A PID is not a job number

`pgrep -af turtlesim` or `ps -ef | grep turtlesim` finds a process ID. `kill 12345` sends a normal stop request; replace `12345` with the real PID.

Pipes and redirections: route program output

```
cd ~/linux_lab
ls | sort              # send output into another command
ls > test.txt         # create or replace a file
ls >> test.txt        # append to the file
ls | tee test.txt  # show and save at the same time
```

Read from left to right

The pipe | connects two commands. `sort` receives the names printed by `ls`.

Warning: > replaces data

It does not ask before overwriting an existing file. » adds lines instead.

Errors and logs: capture evidence

```
cd ~/linux_lab
# Assume the folder missing does not exist
ls missing/test.txt 2> errors.log  # save error output
cat errors.log
tail -n 10 errors.log       # last ten lines
```

Two output streams

Normal output goes to standard output; error messages go to standard error. 2> redirects errors to a file, while > redirects normal output.

Git: track changes in a project folder

```
cd ~/linux_lab
git init
printf '# Linux lab\n' > README.md
git status
git add README.md
git status
```

Read the two status results

The first `git status` shows `README.md` as Untracked. After `git add README.md`, the second status shows *Changes to be committed*: the file is Staged, but no commit exists yet.

`Untracked` ⟶ `Staged` ⟶ `Committed`

Commit and check the branch

`git commit -m "Add README"` records the staged change. Before editing or building someone else's repository, read its README and check the branch with `git branch` or `git status`.

Python projects: isolate dependencies

```
cd ~/linux_lab
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
deactivate
```

What activation changes

The active shell uses the project's Python and `pip`. `deactivate` returns to the prior environment. The folder `.venv/` belongs to this project.

Hint

If `venv` is unavailable, inspect the Ubuntu package `python3-venv`. Install packages only after checking the project's instructions.

Network basics: inspect, then test

```
hostname              # this machine's name
hostname -I           # local IP addresses
ip -brief address     # interfaces and addresses
ping -c 3 127.0.0.1   # test the local network stack
```

What these commands tell you

An address identifies a network interface. The loopback address `127.0.0.1` reaches this computer only; it does not test another machine.

Hint

When two machines must communicate, record both addresses and verify each can reach the other before debugging the application.

SSH: open a terminal on another computer

```
ssh student@192.168.1.42
# After login, on the remote computer:
whoami
hostname
exit
```

Read the connection command

`ssh user@host` opens an encrypted shell on the target. Here `student` is its user; `192.168.1.42` is an example IP.

Check where you are

`whoami` and `hostname` confirm the remote user and machine. `exit` returns to your local terminal.

Hint

The target needs an SSH server and must allow your login. Replace the example user and IP; confirm the target's identity on first connection.

Device files: inspect access before changing it

```
ls /dev/ttyUSB*       # list matching serial devices, if any
ls -l /dev/ttyUSB0    # owner, group, and permissions
groups                # your account's groups
```

Why device files matter

USB serial adapters and other hardware may appear under `/dev`. A program must have permission to open the relevant device file.

Warning: Use the hardware instructions

The correct group or `udev` rule depends on the device. Avoid broad permission changes; inspect the device first.

Troubleshooting: start from the symptom

Symptom	First useful command
Command not found	`command -v name`; check installation and `PATH`.
File not found	`pwd`, `ls`, then `find . -name 'file'`.
Permission denied	`ls -l file`; inspect owner and group.
Program seems stuck	`ps -ef`; identify the process, then use `Ctrl+C` for a foreground program.
Unexpected output	Capture error text with `2> errors.log`; read it with `cat` or `less`.

Hint

Record the exact command, folder, and first error message. That evidence usually points to the next check.

Mini lab: one small project from start to result

```
mkdir -p ~/linux_lab/src
cd ~/linux_lab
printf 'print("hello from Linux")\n' > src/hello.py
python3 src/hello.py | tee run.log
grep 'hello' run.log
```

Expected result

Python prints `hello from Linux`; `tee` saves the same line in `run.log`; `grep` finds that line.

Explain each step

Which command created a folder? Which wrote a file? Which ran a program? Which showed and saved the result?

Linux readiness checklist

1. I can navigate and inspect files: `pwd`, `cd`, `ls`, `find`.
2. I can edit, search, and route text: `nano`, `grep`, `|`, `>`, `tee`.
3. I can install an Ubuntu package with `apt` when needed.
4. I understand shell variables, `source`, and `~/.bashrc`.
5. I can inspect permissions, a process, a log, and a device file.
6. I can create and run the mini lab script and explain its output.

Practice check: can you explain each command?

Read first, then predict what happens

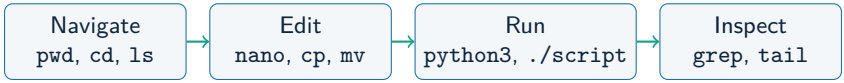
`cd ~` → `mkdir demo` → `cd demo` → `touch test.txt` → `ls`

1. Which command moves you to the home folder?
2. What does `touch` do in this example?
3. What should `ls` display?
4. Which operator appends output: `>` or `»`?
5. Why should you check the target before running `rm -r`?

Expected result

`ls` should show `test.txt`. The » operator appends. `rm -r` can permanently remove its target, so inspect the path and name first.

Quick recap



- Confirm paths and file targets before editing, redirecting, or deleting.
- Use `apt` for system packages and a project environment for local Python dependencies.
- Remember that each terminal has its own variables and running jobs; use `Ctrl+C` to stop a foreground command.
- Diagnose failures through error text, logs, paths, processes, and permissions.

Presenter and contact

Dr. Kaveh Hooshmandi

Primary academic email

k.hooshmandi@arakut.ac.ir

Additional email

Hooshmandi.kaveh@gmail.com